

3^a Série de Problemas

Trabalho realizado por:

Gustavo Costa | N^o 52808

Ian Frunze | N^o 52867

Rafael Pereira | N^o 52880

Turma: **LEIC24D**

Docentes: **Nuno Leite**

Licenciatura em Engenharia Informática e de Computadores

Algoritmos e Estruturas de Dados (AED)

2024 / 2025 - Semestre de Verão

Conteúdo

1	Introdução	2
2	Problema: Detecção de Comunidades em Redes Sociais	3
2.1	Representação do Grafo	3
2.1.1	Estrutura Abstrata do Grafo	4
2.1.2	Implementação <code>GraphStructure<I,D></code>	5
2.1.3	Integração com o Algoritmo	5
2.2	Funcionamento do Algoritmo de Propagação de Etiquetas	6
2.2.1	Lógica do Algoritmo	6
2.3	Observações sobre a implementação	8
3	Exemplo Ilustrativo	9
4	Avaliação Experimental	11
4.1	Teste 1	11
4.2	Teste 2	12
4.3	Conclusão	13
5	Anexos	14
5.1	Classe <code>GraphStrucute<I,D></code>	14
5.2	Função <code>parseGraphFromFile</code>	15
5.3	Função <code>label_propagation</code>	16
5.4	Função <code>main</code>	16

1 Introdução

Num mundo cada vez mais interligado, os grafos assumem um papel fundamental na representação e análise de sistemas complexos, como redes sociais, biológicas ou tecnológicas. Entre os problemas mais relevantes no domínio da análise de grafos encontra-se a detecção de comunidades, cujo objetivo é identificar subconjuntos de vértices fortemente conectados entre si e fracamente ligados ao restante grafo. Estas comunidades podem refletir grupos de amigos em redes sociais, departamentos organizacionais ou até estruturas modulares em redes biológicas.

Neste contexto, o presente trabalho incide sobre a implementação e análise do algoritmo de Propagação de Etiquetas (*Label Propagation Algorithm* - LPA), uma abordagem simples mas eficiente para a deteção de comunidades. O algoritmo parte do princípio de que a identidade de um vértice pode ser influenciada pelas etiquetas dos seus vizinhos, propagando essa informação de forma iterativa até atingir um estado de estabilidade. O LPA apresenta a vantagem de não requerer conhecimento prévio sobre o número de comunidades, além de evidenciar uma boa escalabilidade em grafos de grande dimensão.

O problema em estudo baseia-se num grafo não dirigido, onde os vértices representam usuários de uma rede social e as arestas correspondem às ligações de amizade entre esses mesmos. Através da aplicação do LPA, pretende-se identificar automaticamente os grupos sociais coesos presentes na rede. Para tal, foram implementadas estruturas de dados adequadas à representação do grafo, desenvolvidos métodos para a leitura de ficheiros com dados relacionais e aplicado o algoritmo para produzir uma segmentação em comunidades.

Este relatório descreve detalhadamente a solução desenvolvida, discute as opções de implementação, analisa os resultados obtidos e propõe uma reflexão crítica sobre a eficácia e as limitações do método adotado.

2 Problema: Detecção de Comunidades em Redes Sociais

A detecção de comunidades constitui um dos desafios centrais no estudo e análise de redes complexas. Este problema visa identificar subconjuntos de nós — ou vértices — que se encontram mais fortemente ligados entre si do que com o resto da rede. Estes grupos, denominados comunidades ou *clusters*, representam frequentemente entidades coesas com interesses, funções ou características comuns. Em redes sociais, por exemplo, uma comunidade pode corresponder a um grupo de amigos, colegas de trabalho, membros de uma organização ou usuários com comportamentos semelhantes.

Do ponto de vista computacional, o problema é abordado através de algoritmos que, partindo da estrutura das ligações entre os nós, procuram segmentar o grafo em subconjuntos fortemente interconectados. Uma das estratégias mais simples e intuitivas para este efeito é o algoritmo de Propagação de Etiquetas (*Label Propagation Algorithm* – LPA). Este algoritmo parte de uma ideia base: cada vértice da rede começa por pertencer a uma comunidade distinta, identificada por uma etiqueta única, e, ao longo de várias iterações, adota a etiqueta mais comum entre os seus vizinhos. O processo repete-se até que as etiquetas estabilizem, ou seja, até que os vértices deixem de alterar a sua etiqueta, refletindo, então, uma divisão natural da rede em comunidades.

A simplicidade e eficiência do LPA tornam-no particularmente atrativo para grafos de grande dimensão, onde outras abordagens, mais complexas, tornariam-se demasiado dispendiosas em termos computacionais. Contudo, esta simplicidade também traz limitações, como o risco de produzir diferentes particionamentos consoante a ordem de visita dos vértices, ou de resultar em comunidades de tamanhos muito distintos e pouco homogêneos.

No contexto do presente trabalho, o problema é formulado com base num grafo não dirigido, onde os nós representam utilizadores de uma rede social e as arestas representam ligações de amizade. Os dados são lidos a partir de um ficheiro, no formato `.gr`, onde cada linha descreve as conexões de um utilizador. O objetivo é aplicar o LPA a esta estrutura, identificar automaticamente as comunidades emergentes e apresentar os resultados, permitindo uma leitura clara das agrupações sociais presentes no grafo.

2.1 Representação do Grafo

Para aplicar o algoritmo de detecção de comunidades, é necessário, em primeiro lugar, definir uma representação apropriada da rede social. Neste projeto, a rede é modelada como um grafo não dirigido, em que:

- Cada vértice representa um usuário da rede social;
- Cada aresta representa uma ligação de amizade bidirecional entre dois usuários.

A estrutura de dados utilizada para representar o grafo baseia-se em listas de adjacência, uma abordagem eficiente, tanto em termos de memória como também de desempenho, especialmente adequada para grafos esparsos — ou seja, onde o número de ligações é significativamente inferior ao número máximo possível.

Cada vértice do grafo é identificado por um número inteiro único e contém um campo de dados que armazena a etiqueta comunitária atual (inicialmente, igual ao próprio identificador). As arestas são representadas como pares não dirigidos entre vértices, sendo adicionadas a ambas as listas de adjacência respectivas, garantindo simetria.

A rede é lida a partir de um ficheiro de texto com extensão `.gr`, onde cada linha descreve um vértice e os seus vizinhos diretos no seguinte formato:

```
1 -> 2 4
2 -> 1 3 6
...
```

Código 1: Exemplo de excerto dum ficheiro `.gr` válido

Neste exemplo, o vértice 1 encontra-se ligado aos vértices 2 e 4, e o vértice 2 está ligado a 1, 3 e 6. Durante o carregamento do ficheiro, são adicionados os vértices e as arestas ao grafo, assegurando que as conexões são bidirecionais. Esta representação serve de base à execução do algoritmo de propagação de etiquetas e à análise subsequente das comunidades identificadas.

2.1.1 Estrutura Abstrata do Grafo

Para resolver o problema de deteção de comunidades em redes sociais, é essencial dispor de uma estrutura de dados que represente eficientemente a rede como um grafo. No presente trabalho, foi utilizada uma estrutura baseada em listas de adjacência, construída de forma genérica e modular através da interface `Graph<I, D>`, fornecida no enunciado do exercício 2.

Esta interface define os elementos essenciais de um grafo não dirigido parametrizado por dois tipos genéricos:

- `I`, correspondente ao tipo do identificador de cada vértice;
- `D`, correspondente ao tipo de dados associados ao vértice (neste caso, a etiqueta da comunidade).

A interface `Graph` organiza-se em torno de três componentes principais:

- `Graph.Vertex<I, D>`: representa um vértice, contendo o seu identificador (`id`), os seus dados (`data`) e as suas adjacências;
- `Graph.Edge<I>`: representa uma aresta, com referência ao vértice adjacente;

- Métodos da interface **Graph**: permitem adicionar vértices e arestas, aceder a vértices e arestas específicas, e iterar sobre os vértices existentes.

Esta abordagem permite definir múltiplas implementações da interface **Graph**, promovendo reutilização e separação de responsabilidades. A estrutura concretamente utilizada para a resolução do problema foi implementada na classe **GraphStructure**, descrita na secção seguinte.

2.1.2 Implementação **GraphStructure<I,D>**

A classe **GraphStructure<I, D>** constitui a implementação concreta da interface **Graph<I, D>**, que permite representar grafos não dirigidos com listas de adjacência. Esta estrutura foi construída com o objetivo de oferecer uma base leve, funcional e extensível para a aplicação do algoritmo de propagação de etiquetas. Já as arestas (**Edge<I>**) são objetos simples que referenciam o identificador do vértice adjacente. Estas são armazenadas num **MutableSet** dentro de cada vértice, tendo como vantagem a fácil remoção, adição e alteração das arestas e também garante que não haja arestas repetidas, representando as conexões de forma eficiente.

Relativamente à estrutura interna, os vértices são armazenados num **MutableMap<I, Vertex<I, D>** assim permitindo um fácil e rápido acesso aos vértices e, se necessário, a sua alteração, também garante que não há vértices repetidos, **Vertex** é uma classe interna que implementa **Graph.Vertex<I, D>**. Cada vértice guarda o seu **id**, os dados atuais (etiqueta de comunidade) e um conjunto de arestas (**adjacents**).

Os métodos principais são os seguintes:

- **addVertex(id, data)**: adiciona um vértice, retornando **null** caso já exista.
- **addEdge(id, idAdj)**: adiciona uma aresta entre dois vértices (não dirigida; ambas as direções são tratadas ao nível da leitura do grafo).
- **getVertex(id)** e **getEdge(id, idAdj)**: permitem aceder diretamente aos elementos do grafo.
- **iterator()**: permite iterar sobre todos os vértices existentes no grafo.

Além da interface obrigatória, foi implementado o método adicional **getCommunities()** que percorre todos os vértices, agrupando-os num **MutableMap<D,MutableSet<I>**, onde a etiqueta de cada comunidade representa a chave e a comunidade em si é o valor. Esta função é fundamental para poder observar o resultado após o algoritmo de propagação de etiquetas.

2.1.3 Integração com o Algoritmo

Esta estrutura serve como suporte direto ao algoritmo de propagação de etiquetas. Durante a execução, as etiquetas dos vértices são atualizadas iterativamente através do método **setData**, e as

adjacências são exploradas eficientemente usando `getAdjacencies`. A estrutura garante assim um bom desempenho, mesmo para grafos de dimensão considerável.

2.2 Funcionamento do Algoritmo de Propagação de Etiquetas

O algoritmo de Propagação de Etiquetas (*Label Propagation Algorithm*, LPA) constitui uma das abordagens mais simples e eficazes para a detecção automática de comunidades em redes representadas por grafos. A sua simplicidade algorítmica torna-o particularmente apropriado para redes sociais de larga escala.

2.2.1 Lógica do Algoritmo

O algoritmo `label_propagation` segue uma lógica iterativa. A cada passo, percorre-se o conjunto de vértices e atualiza-se a etiqueta de cada um com base nas etiquetas mais frequentes dos seus vizinhos. O processo repete-se até que nenhuma etiqueta seja alterada numa iteração completa.

O algoritmo inicializa com um `while` onde a condição de paragem é `!stop`, e a primeira linha é colocado o `stop` a `true`, e só depois é efetuada a seguinte lógica do algoritmo:

```
var stop = false

while (!stop) {
    stop = true
```

Código 2: Início do *while*

- Inicialização - Percorre os vértices pertencentes ao grafo e para cada vértice percorrido, queria um `MutableMap<String, Int>` a qual irá ter como chave a *label* e como valor as vezes que essa aparece na vizinhança, em seguida percorre as arestas deste vértice para assim obter as *labels* presentes à sua volta e adicioná-las no `MutableMap`, caso essa *label* não exista no `MutableMap` cria uma entrada desta com o valor 1, caso contrário, apenas adiciona 1 ao valor.

```
graph.forEach { vertice ->
    val count = mutableMapOf<String, Int>()

    // Conta as labels (data) dos vizinhos
    vertice.getAdjacencies().forEach { edge ->
        val adjId = edge.adjacent
        val adj = graph.getVertex(adjId)
        if (adj != null) {
            val adjLabel = adj.data.toString()
            count[adjLabel] = count.getOrElse(adjLabel, 0) + 1
        }
    }
}
```

```
}
```

Código 3: Início do algoritmo

- Seleção da *label* - Após efetuar a contagem das *labels* mais frequentes na vizinhança, irá verificar qual a *label* que tem o valor mais alto, ou seja a *label* mais frequente, caso não exista nenhuma entrada atribui o valor da *label* do próprio vértice iterado, caso exista várias *labels* com o valor igual, é selecionado a que aparece em primeiro no `MutableMap`.

```
val bestLabel = count.maxByOrNull { it.value }?.key ?: vertice.data
```

Código 4: Seleção da *label*

- Critério de paragem - Sabendo qual a *label* mais frequente na vizinhança é verificado se esta é diferente da já existente do vértice, caso sejam diferentes o programa atualiza a *label* do vértice e coloca o `stop` a `false`, assim garantido que efetua-se mais um ciclo, ou seja o algoritmo para quando não houver mais alteração na *label* mais frequente na vizinhança.

```
if (bestLabel != vertice.data){
    vertice.setData(bestLabel)
    stop = false
}
```

Código 5: Condição de paragem

Após a estabilização das etiquetas, cada vértice terá no campo `data` a etiqueta representativa da sua comunidade. A função `getCommunities()` agrupa os vértices com a mesma etiqueta, produzindo um `Map` que associa a cada etiqueta (comunidade) o conjunto de identificadores dos vértices que a integram.

Para facilitar a utilização deste por parte do usuário, foi implementado alguns comandos numa interface CLI utilizando uma biblioteca feita por nós, `CLILib`.

Tutorial:

- Carregar o ficheiro use `loadgraph` ou `lg` e o nome do ficheiro este irá ficar guardado num *buffer*:
- `-> lg graph.gr`
- Se quiser pode guardar esse grafo numa variável, irá guardar o que estiver no *buffer*:
- `-> var grafo`
- Pode visualizar as comunidades do grafo no *buffer*:
- `-> c`

- Ou então visualizar comunidades de um grafo numa variável:
- `-> c grafo`
- E se quiser visualizar o grafo em sim como um todo pode o usar o *viewer* também com a mesma lógica do *buffer* ou da variável:
- `-> v`
- `-> v grafo`
- Irá abrir uma janela *canvas*, para alterar o grafo visualizado basta dar de novo o comando **viewer** com o novo grafo, não aconselho em fechar a janela antes, ainda está em fase beta, se fechar e tentar dar o comando *viewer* de novo, pode congelar a janela.

Nota: Se quiser fazer um *load* de vários grafos sempre que der *run*, pode ir na **main** e utilizar o método **autoExecute**, que recebe o comando e a lista dos ficheiros e onde cada grafo irá ser guardado numa variável com o nome do ficheiro, como por exemplo, um ficheiro **graph1.gr** irá ser guardado numa variável **graph1**; isto tem que ser feito antes de dar *run* ao **App**.

2.3 Observações sobre a implementação

- A etiqueta mais frequente entre os vizinhos é determinada usando **maxByOrNull**, o que garante uma escolha simples mas eficaz.
- O algoritmo assume que o grafo é não dirigido, e por isso adiciona as arestas em ambas as direções no momento da leitura do ficheiro **.gr**.
- A abordagem utilizada é totalmente determinística dentro de uma execução, mas o resultado pode variar com a ordem dos vértices

3 Exemplo Ilustrativo

Para uma melhor compreensão do funcionamento do algoritmo de Propagação de Etiquetas, é útil analisar a sua aplicação a um exemplo concreto, de pequena dimensão, mas representativo da lógica do processo. Suponha-se o seguinte grafo não dirigido, exemplo fornecido no enunciado da série, que representa uma rede social simplificada composta por doze usuários. A estrutura da rede é descrita no formato `.gr`, tal como suportado pela função `parseGraphFromFile`, com o seguinte conteúdo:

```
1 -> 2 4
2 -> 1 3 6
3 -> 2 4 9
4 -> 1 3
5 -> 6 8
6 -> 5 7 2
7 -> 6 8 11
8 -> 5 7
9 -> 3 10 12
10 -> 9 11
11 -> 10 12 7
12 -> 9 11
```

Código 6: Exemplo de conteúdo do ficheiro `.gr`

Este ficheiro define um grafo com três grupos fortemente conectados: $\{1, 2, 3, 4\}$, $\{5, 6, 7, 8\}$ e $\{9, 10, 11, 12\}$. Embora existam algumas ligações entre grupos, cada subconjunto forma uma comunidade coesa. Esta estrutura é ideal para observar a dinâmica do algoritmo de Propagação de Etiquetas.

Na fase inicial, cada vértice recebe uma etiqueta igual ao seu próprio identificador. Por exemplo, o vértice 1 tem etiqueta “1”, o vértice 2 tem etiqueta “2”, e assim sucessivamente. À medida que o algoritmo avança, os vértices atualizam iterativamente as suas etiquetas de acordo com as etiquetas mais frequentes dos seus vizinhos. Esta atualização tende a propagar as etiquetas dominantes dentro de cada grupo de vértices densamente interligados.

Numas primeiras iterações, observam-se alterações frequentes de etiquetas, uma vez que ainda não se atingiu um estado de equilíbrio. Por exemplo, o vértice 2, ligado a 1, 3 e 6, poderá adotar rapidamente a etiqueta mais comum entre esses, assumindo eventualmente a de 1 ou de 3, consoante a convergência local. Um comportamento semelhante verifica-se nos vértices dos outros grupos.

Com o decorrer das iterações, o sistema estabiliza. As etiquetas deixam de mudar porque todos os vértices passaram a partilhar a mesma etiqueta dominante do seu grupo. No caso deste exemplo, os vértices 1, 2, 3 e 4 acabam por partilhar a mesma etiqueta, por exemplo, “2”. O mesmo sucede para os conjuntos $\{5, 6, 7, 8\}$ e $\{9, 10, 11, 12\}$, que podem convergir para “6” e “10”, respetivamente.

O resultado final consiste num conjunto de comunidades bem definidas, cada uma identificada por uma etiqueta única e composta pelos vértices que adotaram essa etiqueta. A execução do método

`getCommunities()` após a convergência devolve uma estrutura semelhante a:

```
{2=[1, 2, 3, 4], 6=[5, 6, 7, 8], 10=[9, 10, 11, 12]}
```

Código 7: Exemplo de output gerado após execução do algoritmo

Este resultado mostra claramente três comunidades coesas, o que reflete adequadamente a estrutura latente do grafo original. A visualização final gerada por **GraphViewer** também permite observar graficamente estes agrupamentos, facilitando a validação e análise dos resultados.

Este exemplo demonstra de forma eficaz a capacidade do algoritmo para detetar comunidades com base apenas na topologia da rede, sem necessidade de qualquer supervisão ou informação externa. Mesmo num exemplo pequeno, o comportamento do algoritmo evidencia os seus princípios fundamentais: propagação local de informação, convergência descentralizada e formação emergente de *clusters*.

4 Avaliação Experimental

Para avaliar a eficiência do nosso algoritmo, realizaram-se dois tipos de testes:

4.1 Teste 1

Aqui utilizámos grafos onde o total das comunidades é igual ao valor total de nós por cada comunidade, e obteve-se o seguinte resultado:

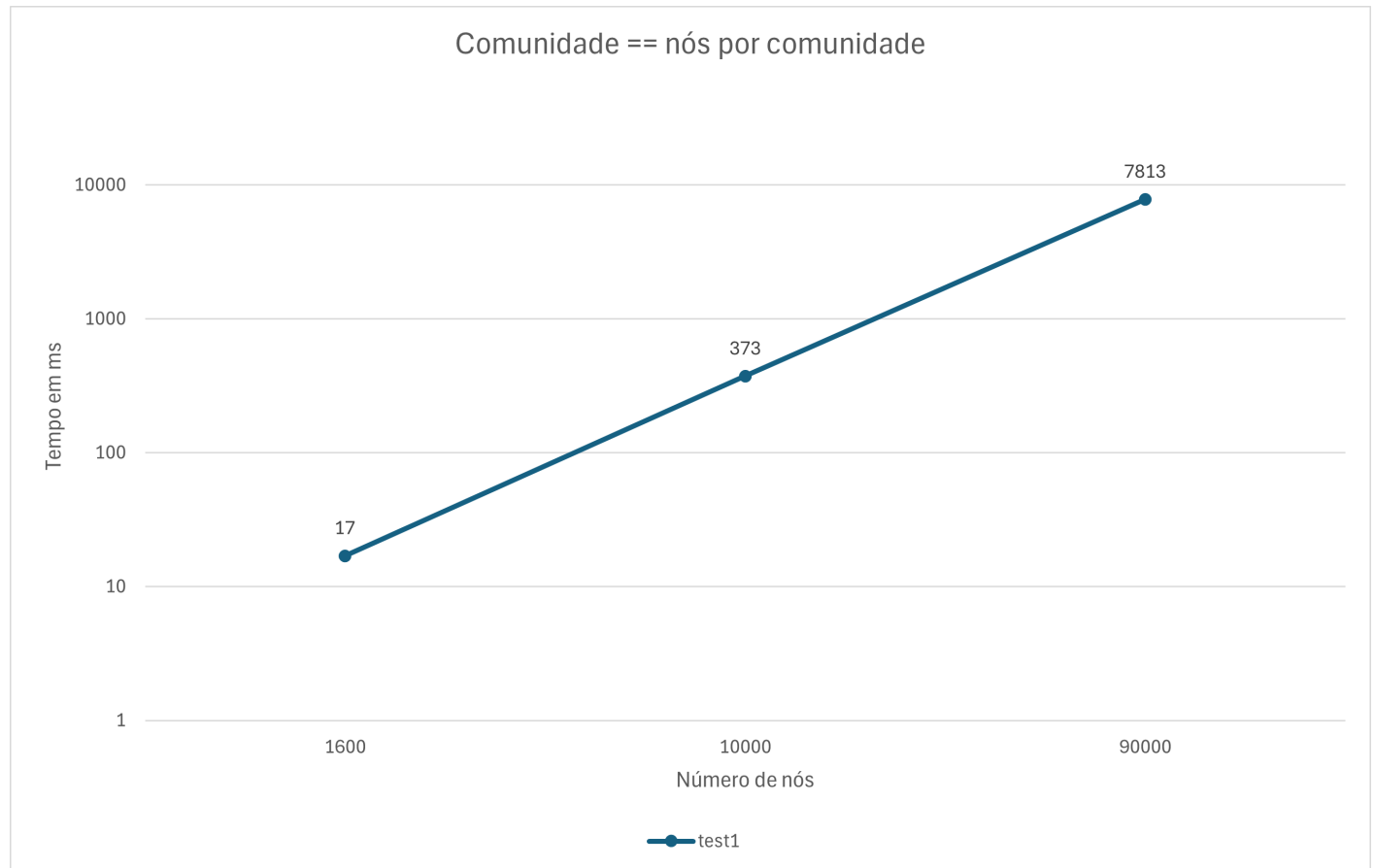


Figura 1: Gráfico relativo ao `test1`

Ao observar o gráfico acima, conclui-se que há uma linearidade bem definida, confirmando que o tempo é diretamente proporcional com o número total de nós do grafo.

- Para 40 comunidades com 40 nós cada, totalizando 1600 nós, demorou cerca de 17ms;
- Já para 100 comunidades com 100 nós cada, totalizando 10 mil nós, demorou cerca de 373ms;
- Por fim, para 300 comunidades com 300 nós cada, totalizando 90 mil nós, demorou cerca de 7813ms = 7,813s.

Ao analisar a forma como o algoritmo funciona, consegue-se prever que para este tipo de teste, à medida que aumentamos o número de comunidades, e consequentemente, o número de nós por cada comunidade, o tempo para gerar o grafo também é aumentado.

Para efeitos teóricos, se considerarmos 5 mil comunidades com 5 mil nós cada uma, expecta-se que o tempo demorado para gerar o grafo correspondente seja de aproximadamente 2 minutos (130 segundos). Um valor muito aceitável para gerar um grafo com 25 milhões de nós.

4.2 Teste 2

Para este teste, quisemos observar o comportamento do algoritmo quando mantemos o número total de nós constante, mas variamos a relação entre o número de comunidades e o número de nós por comunidade.

Por exemplo, no primeiro ficheiro, temos um total de 400 nós. No gráfico representado a verde, temos 10 comunidades com 40 nós cada. Já na linha azul, mantemos os mesmos 400 nós totais, mas invertemos as variáveis: temos 40 comunidades com 10 nós cada.

Mostramos em seguida o gráfico relativo ao segundo teste:

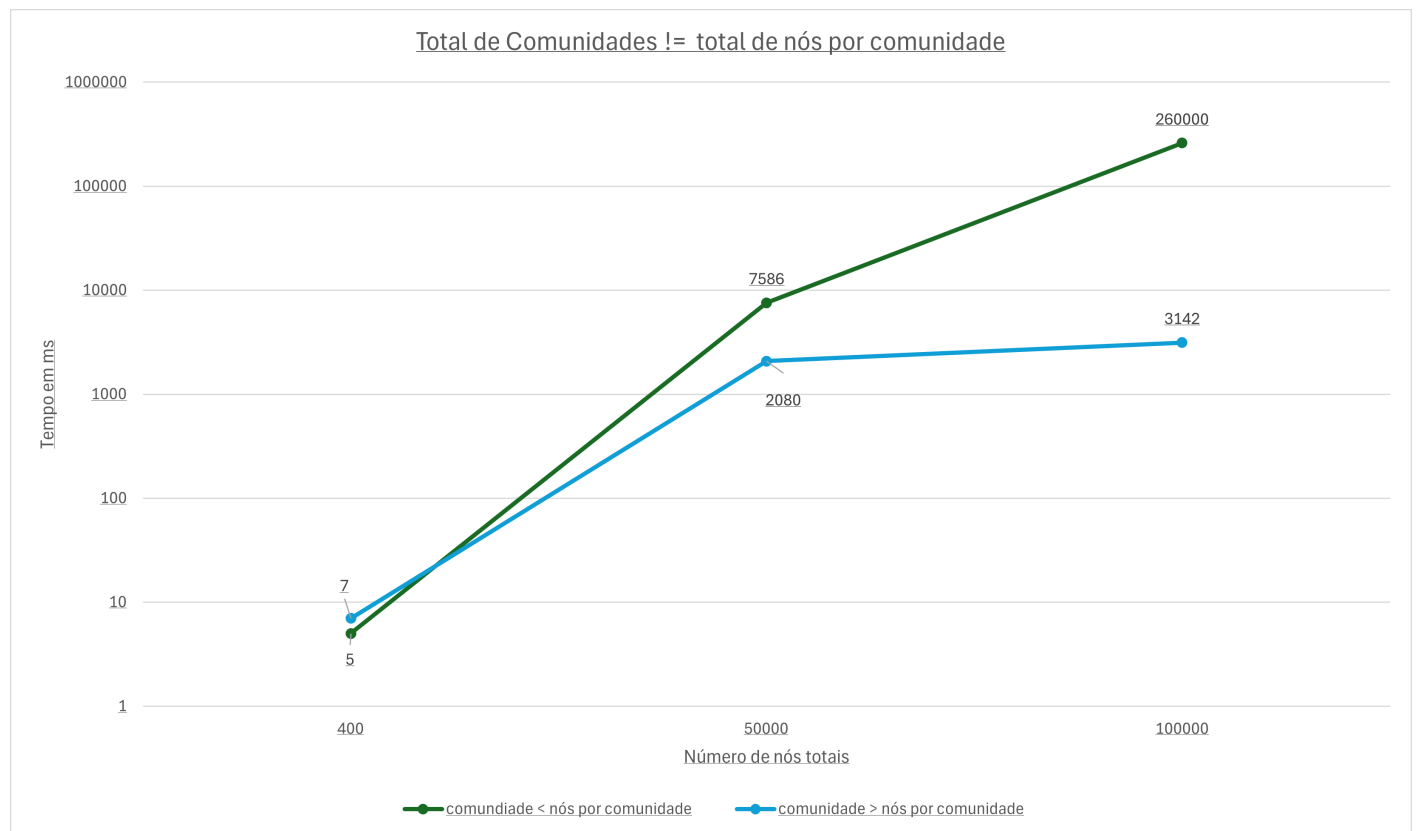


Figura 2: Gráfico relativo ao teste2

Neste teste, observa-se e conclui-se algo interessante.

Ao comparar as duas linhas, nota-se que já não tomam a mesma evolução. A linha verde, relativa às comunidades com mais nós, tomou um crescimento mais acentuado relativamente à linha azul que representa as comunidades com menos nós.

4.3 Conclusão

Observando os resultados conseguimos concluir que quanto maior o número de comunidades e menor o número de nós por comunidade, sendo isto, comunidades melhor definidas e em maior quantidade, temos uma melhor eficiência geral, já em grafos com poucas comunidades em relação aos nós por comunidade, ou seja, comunidades muito densas e pouco definidas entre si temos uma maior ineficiência, podendo observar, nitidamente, essa diferença na última estatística da Figura 2, onde este tem um total de 100 mil nós totais; Na linha verde temos 100 comunidades e 1000 nós por comunidade, já na linha azul temos 1000 comunidades e 100 nós por comunidade. A linha azul (3,142s) é cerca de 83 vezes mais eficiente relativamente à linha verde (260s), para o mesmo número de nós totais, concluindo essa divergência relativamente à forma como estão definidas as comunidades do grafo, e também, o processo de análise do algoritmo usado por nós.

5 Anexos

5.1 Classe GraphStrucute<I,D>

```
class GraphStructure<I, D>: Graph<I, D> {
    override val size: Int
        get() = vertices.size

    val vertices = mutableMapOf<I, Vertex<I,D>>()

    class Vertex<I,D>(override val id: I, override var data: D): Graph.Vertex<I,D> {
        private val adjacents = mutableSetOf<Graph.Edge<I>>()

        override fun setData(newData: D): D {
            val temp = data
            data = newData
            return temp
        }

        override fun getAdjacencies(): MutableSet<Graph.Edge<I>> = adjacents
    }

    class Edge<I>(override val id: I, override val adjacent: I): Graph.Edge<I>

    override fun addVertex(id: I, d: D): D? {
        val existe = vertices.putIfAbsent(id, Vertex(id,d))
        return if (existe == null) d else null
    }

    override fun addEdge(id: I, idAdj: I): I? {
        val vertex = vertices[id] ?: return null
        vertex.getAdjacencies().add(Edge(id,idAdj))
        return id
    }

    override fun getVertex(id: I): Graph.Vertex<I, D>? = vertices[id]

    override fun getEdge(id: I, idAdj: I): Graph.Edge<I>? {
        val vertice = vertices[id] ?: return null
        vertice.getAdjacencies().forEach {if (it.adjacent == idAdj) return it}
        return null
    }

    override fun iterator(): Iterator<Graph.Vertex<I, D>> = vertices.values.
    iterator()
}
```

```
fun getCommunities(): MutableMap<D,MutableSet<I>> {
    val communities = mutableMapOf<D,MutableSet<I>>()
    vertices.forEach { (_,vertice) ->
        val label = vertice.data
        val community = communities.getOrPut(label) {mutableSetOf()}
        community.add(vertice.id)
    }
    return communities
}
```

Código 8: Implementação da classe GraphStrucute<I,D> em *Kotlin*

5.2 Função parseGraphFromFile

```
fun parseGraphFromFile(file: String): GraphStructure<Any,Any> {
    val graph = GraphStructure<Any,Any>()
    var lines = emptyList<String>()
    try {
        lines = File(file).readLines()
    } catch (e: Exception) {
        println("Erro ao ler arquivo: ${e.message}")
    }
    if (!lines.isEmpty()) error("Ficheiro vazio")

    lines.forEach { it ->
        val line = it.split("->")
        val vertice = line[0].replace(" ", "")
        graph.addVertex(vertice, vertice)

        line[1].split(" ").forEach {adj->
            if (adj != "") {
                graph.addEdge(vertice, adj)
            }
        }
    }
    return graph
}
```

Código 9: Implementação da função parseGraphFromFile em *Kotlin*

5.3 Função label_propagation

```
fun label_propagation(graph: GraphStructure<Any, Any>) {
    var stop = false

    while (!stop) {
        stop = true

        graph.forEach { vertice ->
            val count = mutableMapOf<String, Int>()

            // Conta as labels (data) dos vizinhos
            vertice.getAdjacencies().forEach { edge ->
                val adjId = edge.adjacent
                val adj = graph.getVertex(adjId)
                if (adj != null) {
                    val adjLabel = adj.data.toString()
                    count[adjLabel] = count.getOrDefault(adjLabel, 0) + 1
                }
            }

            val bestLabel = count.maxByOrNull { it.value }?.key ?: vertice.data
            if (bestLabel != vertice.data){
                vertice.setData(bestLabel)
                stop = false
            }
        }
    }
}
```

Código 10: Implementação da label_propagation, em *Kotlin*

5.4 Função main

```
fun main() {
    //Notinha esta em beta o viewer se quiser alterar o grafo que aparece na janela
    canvas
    //basta dar o comando viewer com o novo grafo,
    //se fechar a janela provavelmente vai congelar o viewer
    val app = App
    app.autoExecute(LoadGraph, listOf("graph.gr", "graph1.gr", "graph2.gr"))
    app.run()
}
```

Código 11: Implementação da main, em *Kotlin*